

Overview

StockShark is a web-based application that empowers users to view, compare, and manage stock portfolios (or watchlists). The application integrates with external market data APIs (e.g., Alpha Vantage) to retrieve real-time and historical stock data. It provides features such as:

- User Registration and Authentication: New users can register, log in, and manage their profile.
- Market Overview and Stock Comparison: Users can view market data, compare stocks over custom date ranges, and see graphical representations using Chart.js.
- Watchlist/Portfolio Management: Users can create and manage portfolios (watchlists) where they add their favorite or frequently traded stocks.
- Data Persistence: All user and portfolio data are persisted in a MySQL database using a robust schema with proper referential integrity.
- Interactive, Modern UI: A dark-themed interface with responsive design and interactive charts for a smooth user experience.

Architecture & Design:

The StockShark project follows a 5-tier architecture with the following layers:

Presentation Layer:

JSP Pages and HTML/CSS/JavaScript: User interface is rendered by JSPs that incorporate JSTL, custom EL expressions, and utilize CSS for styling. Client-side interactivity is provided via JavaScript and Chart.js for dynamic charting.



Controller Layer:

Servlets: Java servlets handle HTTP requests and responses. Notable servlets include LoginServlet, RegistrationServlet, CompareStocksServlet, PortfolioServlet, and WatchlistServlet. They are mapped using annotations (e.g., @WebServlet). They manage request/response cycles, encapsulate user interactions and forward data to the view fulfilling user stories such as viewing personalized market data and portfolios.

Business Logic Layer:

The business logic is organized into compound components like User_Management_Compound and Data_Handling_Component. This abstraction facilitates meeting complex user interactions (e.g., adding portfolios, updating charts) while keeping the controllers lean.

Data Access Layer:

Database Access Classes: Classes like PortfolioDAO encapsulate database access. They run SQL queries using JDBC with proper resource management (try-with-resources) and manage transactions. Instead of a DAO, our project uses a Database Client class to handle all persistence operations. This ensures that queries and updates such as retrieving portfolio details by joining portfolios with ‘portfolio items’ are centralized. This design was chosen to maintain flexibility and consistency with our overall application workflow.

Integration Layer:

The application integrates with external APIs (such as Alpha Vantage) to fetch real-time stock data. User stories requiring updated market information are supported through API calls managed via the Database Client. The API responses (JSON) are parsed and converted into domain objects (like StockData).

Stock Shark

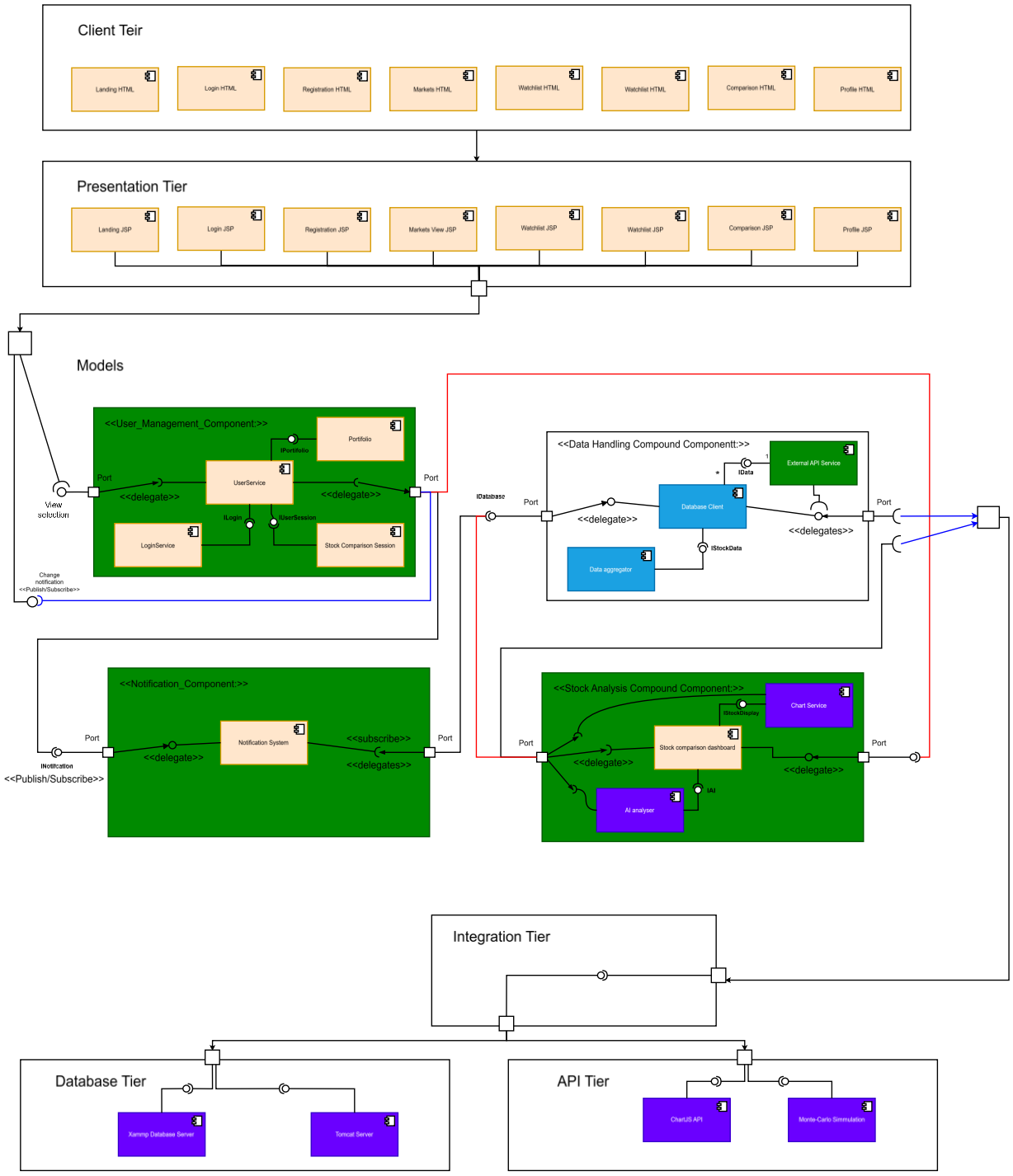
THE SOFTWARE ARCHITECTURE & DESIGN

5 Tier Architecture Component Diagram:



Stock Shark

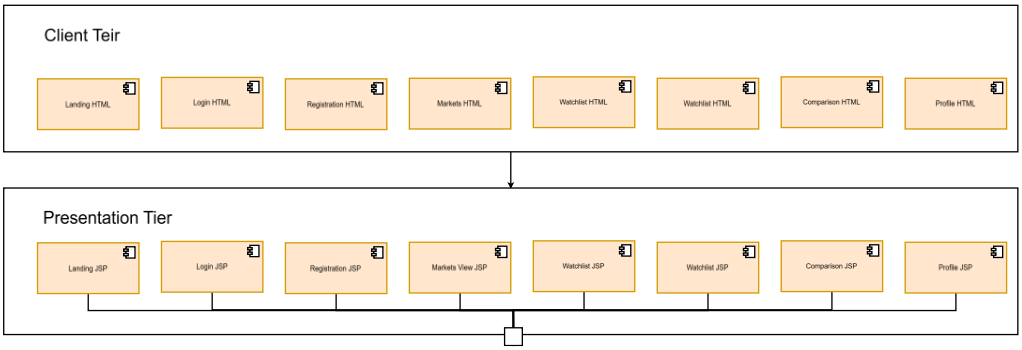
THE SOFTWARE ARCHITECTURE & DESIGN



Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

This solution uses the 5-tiered architecture.



Tier 1 – Client Tier

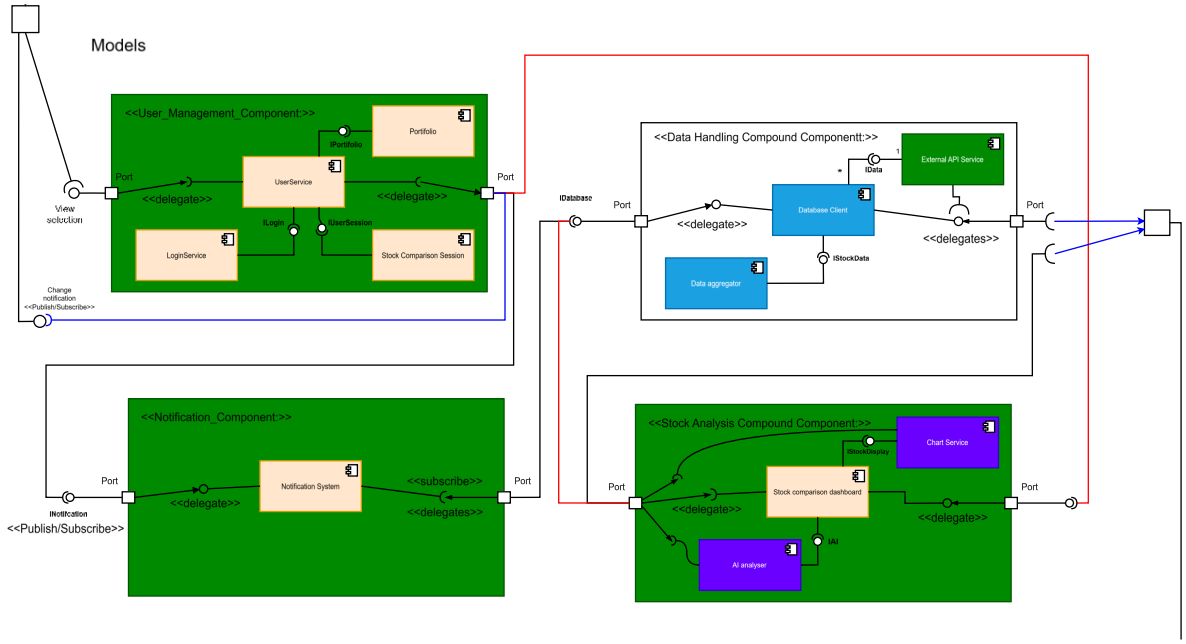
To provide a complete design, we include a client tier (the first tier), which contains the UI elements that users employ to interact with the rest of the system.

Tier 2 – Presentation Tier

The first tier then interacts with the presentation tier, which contains our servlets. These servlets decide which of the models should handle each user interaction. They do so by connecting to a dedicated port in the user components that serves as the main entry point to the **User_management_Component**. This component is responsible for handling user-facing requests such as login, portfolio actions, and starting stock-comparison sessions.

Stock Shark

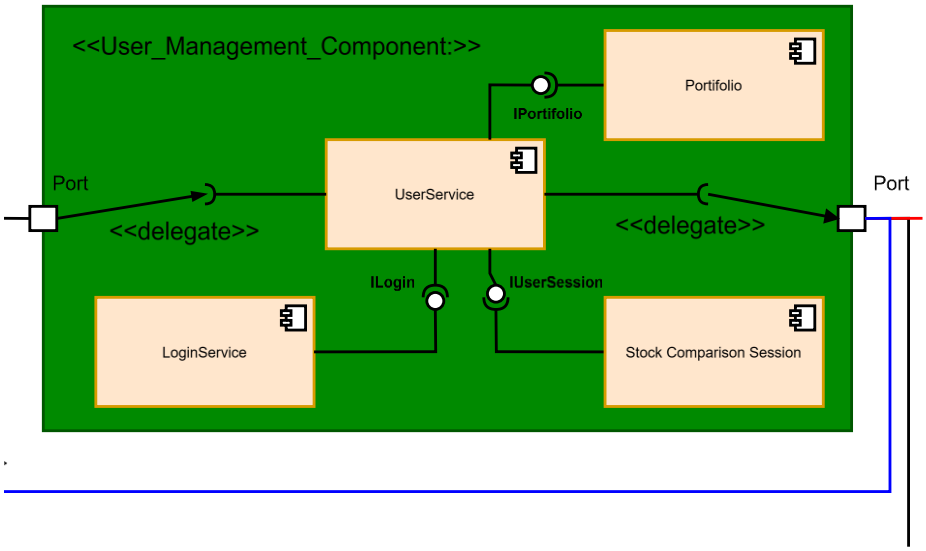
THE SOFTWARE ARCHITECTURE & DESIGN



Tier 3 – Business Logic Tier/ Models

The business logic is encapsulated in four client Compound components:

- **User_Management_Component** – handles front-facing user requests that come from the servlets.
- **Data_Handling_Compound_Component** – manages all incoming and outgoing data packets and how they are manipulated.
- **Stock_Analysis_Compound_Component** – provides custom-coded charts and refined mathematical comparison calculations.
- **Notification_Compound_Component** – sends notifications to the user and listens for changes within the Data_Handling component



This compound component has four sub-components:

- **Portfolio:** responsible for creating Portfolio objects for the user and contains methods for accessing and storing them.
- **LoginService:** responsible for providing methods that can be used to authenticate and log users in and out of sessions on the platform.
- **StockComparison:** responsible for initiating and terminating comparison sessions.
- **UserService:** responsible for hosting the ports that other external components can connect to and also to communicate with other internal components.

The UserService has three interfaces:

- **IPortfolio:** exposes functionality that can be used to manage a portfolio, such as adding, removing, and updating.
- **ILogin:** exposes functionality related to login, logout, and registration.
- **IUserSession:** exposes functionality that can be used to start or stop stock-comparison sessions.

Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

<<Interface>> **IPortfolio**
+ Map<String, List<String>> getPortfolio(String porfolioID); + Boolean updatePortfolio(String email, String symbolA, String SymbolB); + Boolean removePortfolio(String user, Portfolio portfolio);

getPorfolio: returns a dictionary whose key is the PortfolioID and whose value is a list of Stock Symbols for the specified portfolio.

Update/Remove Portfolio: returns **true** if insertion into the database succeeds, or **false** if it fails.

<<Interface>> **ILogin**
+ Boolean registerUser(String username,String email, String password, Data_Handling_Component dh); + Boolean loginUser(String email, String password, Data_Handling_Component dh); + Boolean logoutUser(String email, Data_Handling_Component dh);

Login/Logout/Register: returns **true** if insertion into the database succeeds, or **false** if it fails.

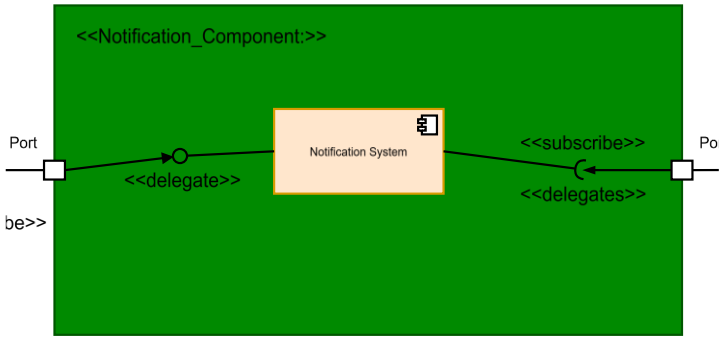
<<Interface>> **IUserSession**
+ void startSession(List<String> stockSymbols); + void endSession(); + boolean isSessionActive(boolean session);

IsSessionActive: returns **true** if the user is currently in an active comparison session, or **false** otherwise.

he **User_Management_Compound_Component** communicates directly with two other compound components:

- Notification_Compound_Component
- Stock_Analysis_Compound_Component.

The **Notification_Compound_Component** offers a single provided interface, **iNotification**, which exposes publish/subscribe functionality so users are informed of the latest system changes. When a user interacts, they can update the notification system, enabling it to track any additional data to which it is subscribed.



<<Interface>> **INotification**
+ void sendNotification(String message); + List<String> viewNotifications(String userID);

Send Notifications: collects and sends new symbols or portfolio updates to the system so that it will now listen for updates within the database.

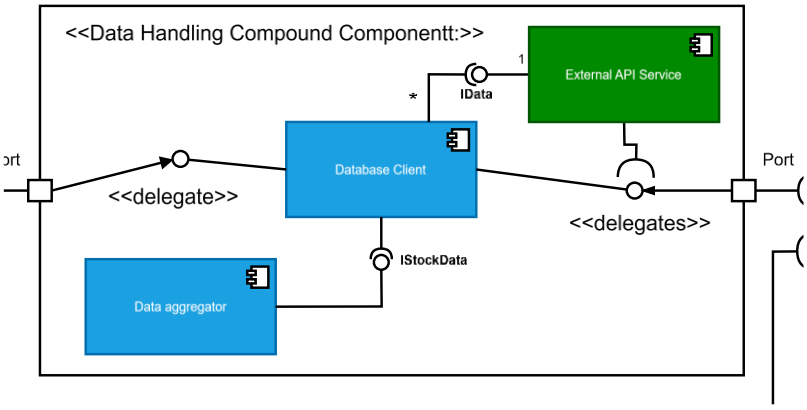
View Notifications: returns a list of symbols reflecting the latest system changes that have occurred within the user's saved portfolio items.

This compound component contains a single sub-component, **Notification System**. It encapsulates the core functionality for updating notifications in — and retrieving them from — the database. The sub-component exposes two ports for external communication:

- **Notification_ComponentPort:** provides notification functionality to the User_Management_Compound_Component.
- **getDataHandlingComponentPort:** connects to the data-handling compound components that expose database-reading functionality through the required interface IDatabase.

<<Interface>> **IDatabase**
+ Database connectToDatabase();

Once the Data Handling Component Port is invoked, it exposes the functionality of the Data Handling Compound Component by connecting to the delegated port getDataHandlingComponentPort hosted within the DatabaseClient subcomponent.



The Database Client has two required interfaces:

- **iData:** interface provided by the External API Service module; exposes functionality used to communicate with APIs outside our system. It is defined as a one-to-many relationship because multiple methods for different API providers can be made available to our Database Client.

- **iStockData:** interface provided by the Data Aggregator; exposes functionality that translates data pulled from our External API Service into a form usable by our application.

<<Interface>> **IData**
+ JSONObject getStockData(String symbol) throws IOException, InterruptedException; + JSONObject getStockDataByDateRange(String symbol, String startDate, String endDate) throws IOException, InterruptedException; + JSONObject getTrendingMarketData(String symbol) throws IOException, InterruptedException;

getStockData: returns stock data in JSON format for a defined date range if a date is not specified.

getStockDataByDateRange: returns stock data from AlphaVantage in JSON format for a specified date range placed by the user.

getTrendingMarketData: returns data from AlphaVantage for a defined set list of symbols.

<<Interface>> **iStockData**
+ List<StockData> formatData_from_AlphaVantage(JSONObject jsonData);

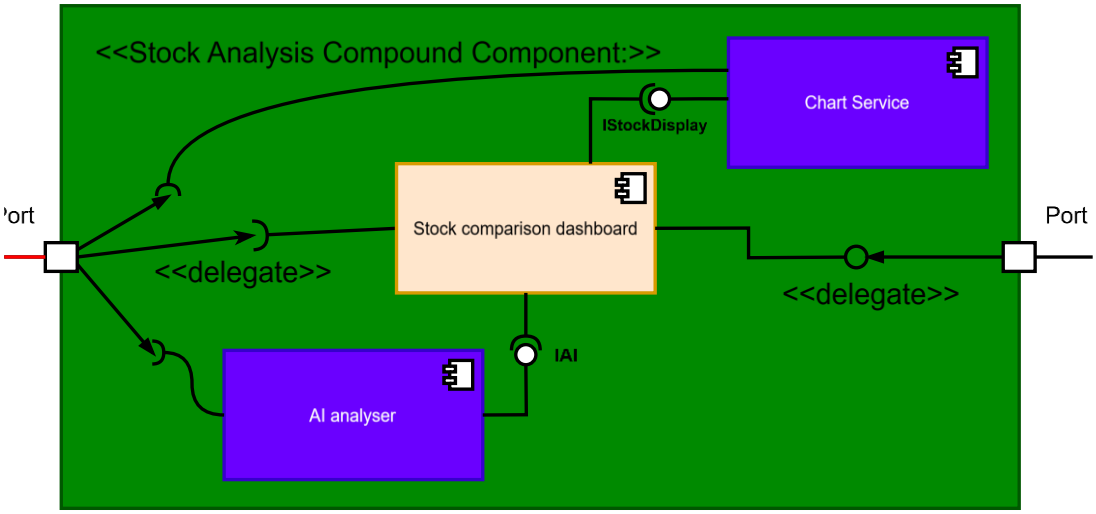
formatData_from_AlphaVantage: receives the data from Alpha Vantage and returns a List of Stock objects that can be used by our system classes.

On the other hand, the UserService could take a different route to communicate with the database by using its required interface:

- **StockAnalysis:** exposes the functionality of the Stock Analysis Compound Component by connecting to the delegated port hosted within the Stock Comparison Dashboard.

Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN



<<Interface>> iStockDisplay
public void GenerateAreaChart(List<String> stockData); public void GenerateAreaChartByDateRange(List<String> stockSymbols,String date1, String date2);

GenerateAreaChart: retrieves data from the database and formats it into an array of values suitable for plotting a graph.

GenerateAreaChartByDateRange: performs the same operation but filters the data so it returns values for a specific date.

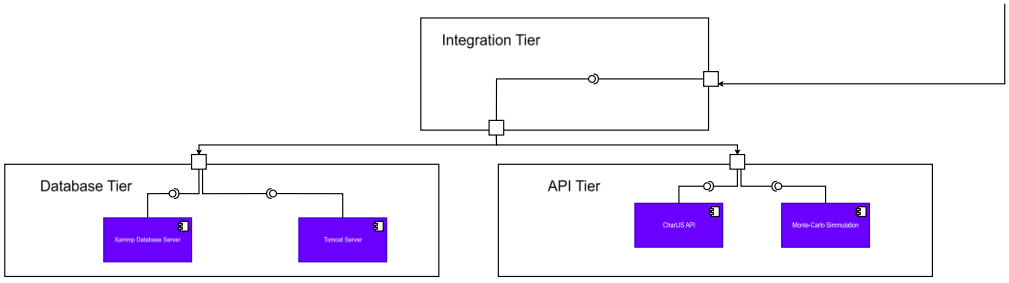
The Stock Analysis Compound component had three sub components:

- **ChartService:** requires the Chart .js API and provides usable chart templates to the Stock Comparison Dashboard.
- **AI Analyser:** requires the OpenAI API and supplies summarized analysis and insights to the Stock Comparison Dashboard.

- **StockComparison Dashboard:** encapsulates functionality for users to create diagram comparisons and has two interfaces:
    - **IStockDisplay** – already detailed.
    - **IAI** – this interface is not yet implemented.

The stock comparison dashboard has two delegated ports that is responsible for providing communication with the external components:

- **getDataHandlingComponentPort:** creates a required interface for connecting to the Data Handling Component.
- **getStockAnalysisComponentPort:** provides external components with access to the internal methods of the Stock Analysis Compound Component.



Tier 4 – Integration Tier: exposes a localhost port that allows the system to connect to the external services located in Tier 5.

Tier 5 – External Services Tier: contains all services external to the system, including:

- the **Database tier**, which consists of our Tomcat server and XAMPP, and
- the **API tier**, which consists of the AlphaVantage API and ChartJS API.

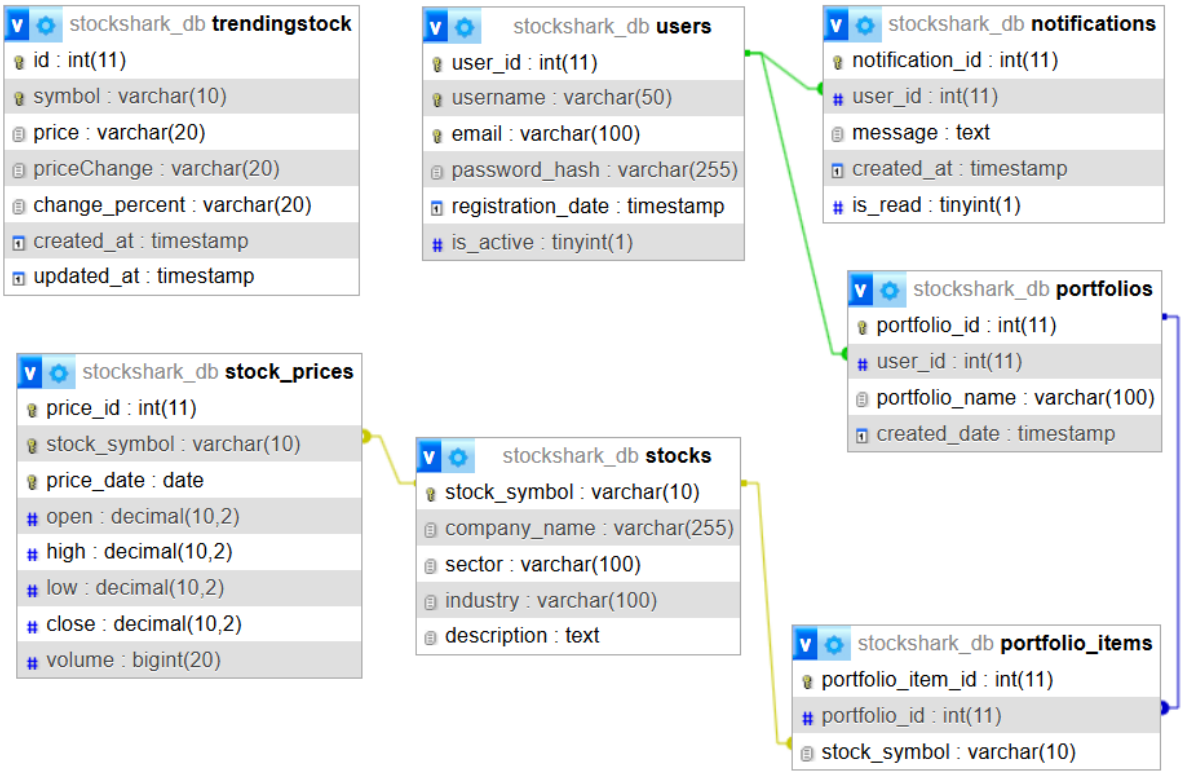
Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

How is this achieved?

The Database:

The database schema was designed to enforce data integrity and performance while supporting key functionalities like user management, stock data retrieval, and portfolio tracking. This design ensures that critical user data is stored securely and reliably, directly addressing user stories for privacy and accessibility.





Key Tables:

- **users:** Stores user credentials and profile details.
- **stocks:** Contains static information about stocks.
- **stock_prices:** Maintains historical (and potentially real-time) pricing information.
- **portfolios:** Saves user-defined portfolios or watchlists.
- **portfolio_items:** Associates specific stocks with a given portfolio.
- **notifications:** (Planned) Will store user notifications.

Deployment:

The deployment environment was selected to balance ease of development with production readiness. XAMPP is used as a local MySQL development server, and Apache Tomcat serves as the web/application server, ensuring broad compatibility and robust performance.

Environment Details:

- **Database Server:**
  MySQL provided through XAMPP.
- **Application Server:**
  Apache Tomcat.
- **Build Tools:**
  Maven is used for building and packaging the application as a WAR file.
- **Configuration:**
  Database configuration details (host, port, database name, user credentials) are managed centrally via the DatabaseConfig class.

Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

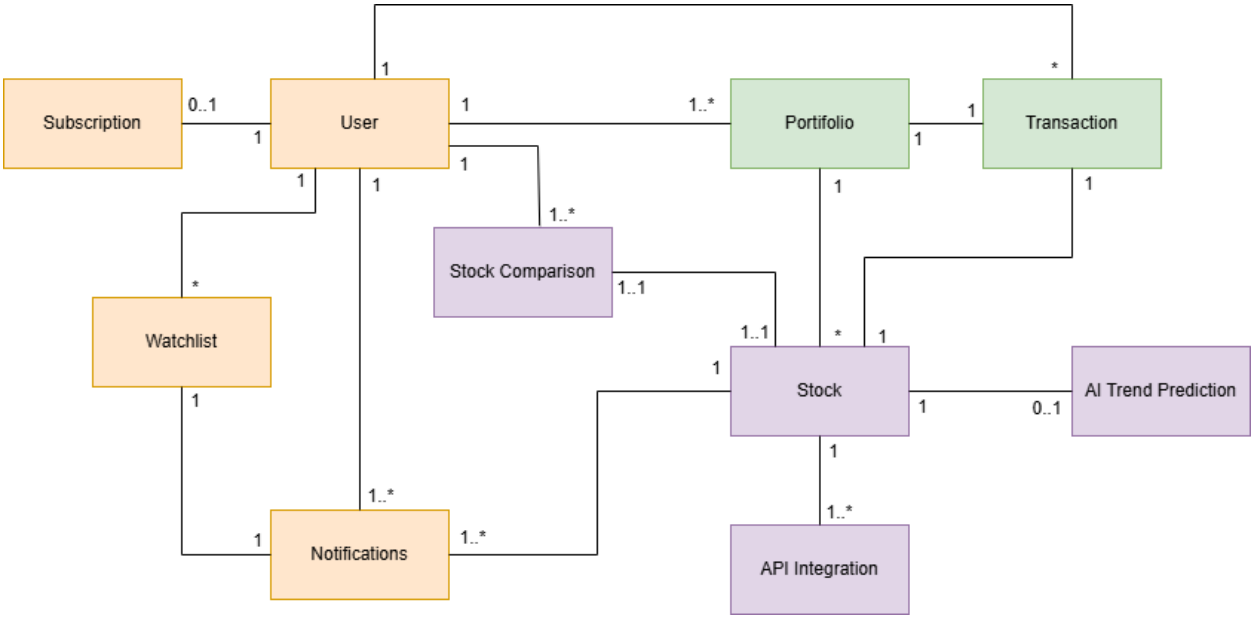
Building The Architecture

StockShark's architecture is built upon the core principles of Clean Architecture, which aims to create systems that are independent of frameworks, testable, and maintainable. This design follows the SOLID principles, making the system strong and easy to change.

- Single/Closed Principle (OCP): StockShark is designed to be open for extension but closed for modification. New features are added by introducing new code, rather than altering existing, stable code.
- Dependency Inversion Principle (DIP): High level modules, such as use cases that implement business logic, do not depend on low level modules, such as data access layers. Instead, low level modules depend on abstractions defined by high level modules.

By separating these layers, we create a system that's flexible, maintainable, and easy to understand. This makes it easier for our team to add new features, fix bugs, and adapt to changing requirements, ultimately providing you with a better StockShark experience.

Business Concept Model

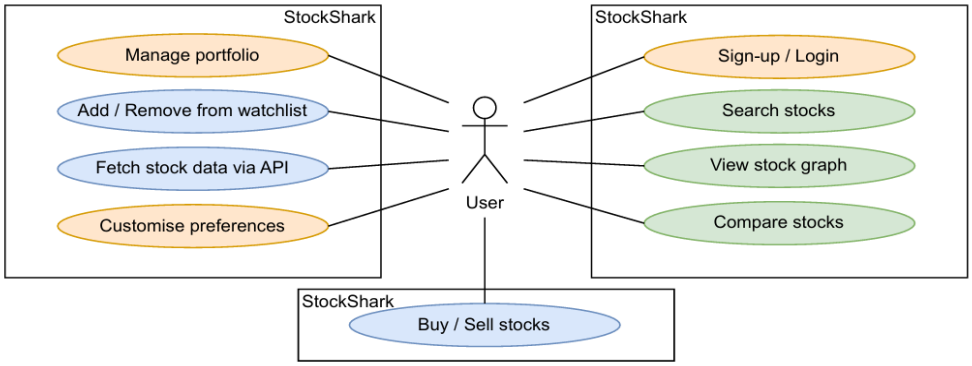


Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

The Business Concept Model is structured to provide a user-centric, scalable, and modular system for managing stock investments and transactions. At its core, the User entity connects to essential functions like portfolios, transactions, subscriptions, watchlists, and notifications, ensuring seamless interaction with the platform. The Stock entity is deeply integrated with real-time market updates via API integration, AI trend predictions, and stock comparisons, allowing for data-driven decision-making. The model also enforces flexibility, allowing users to manage multiple portfolios and transactions while ensuring that each action is properly tracked. The relational structure supports future expansion, such as adding new AI models, additional investment tools, or automation, without disrupting existing functionalities. By balancing functionality and scalability, this design enables a robust financial system that caters to both beginner and advanced investors.

Use Case Model



The StockShark Use Case Diagram is structured to provide a clear and user-centric experience, dividing functionalities into three main categories: portfolio management, stock exploration, and trading transactions. Users can manage their investments by adding or removing stocks from their watchlist, customizing preferences, and fetching real-time stock data via API. The system also supports stock research through features like stock search, comparison, and historical graph visualization. Additionally, the buy/sell stocks functionality ensures seamless trading operations. The inclusion of sign-up/login enhances security and access control. This structured approach ensures scalability, allowing for future enhancements like AI-driven insights or automated trading while maintaining an intuitive and efficient user workflow.

Stock Shark

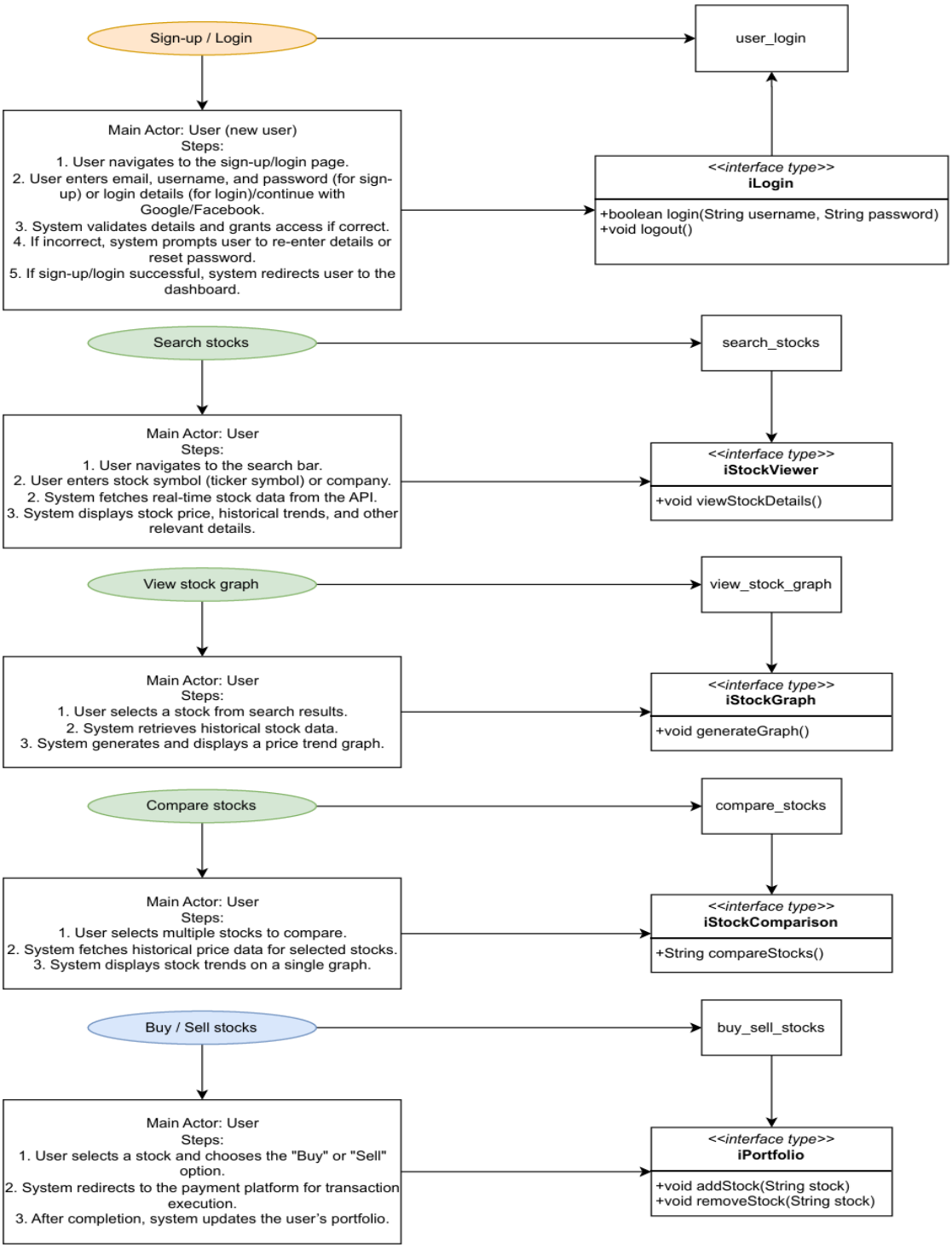
THE SOFTWARE ARCHITECTURE & DESIGN

System Interfaces



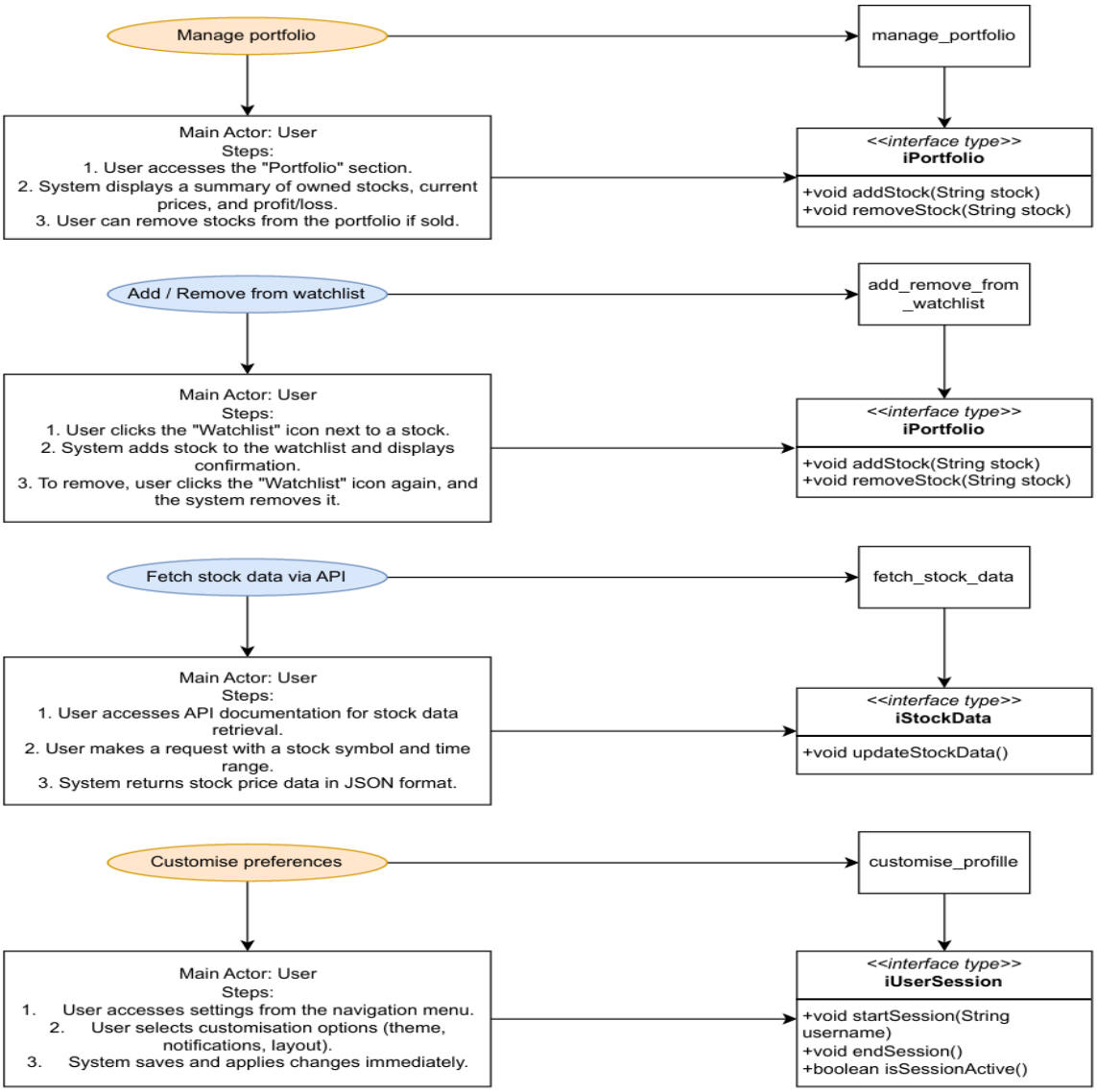
Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN



Stock Shark

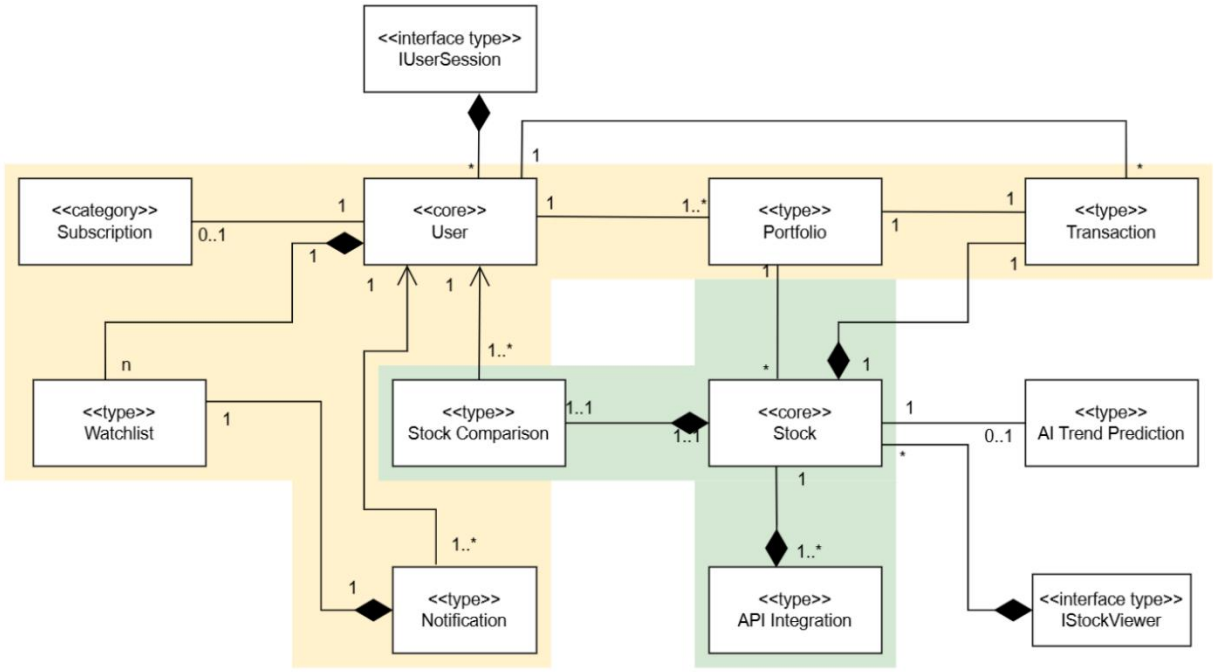
THE SOFTWARE ARCHITECTURE & DESIGN



Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

Business Type Model

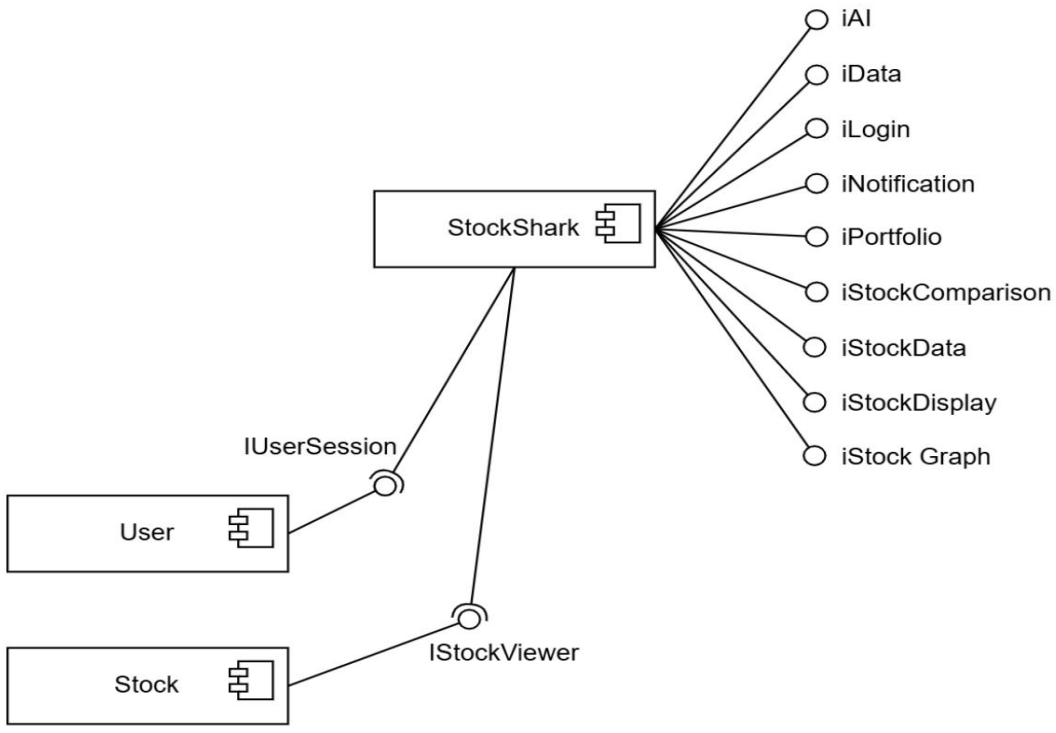


The Business Type Model in the diagram is structured to represent a scalable and modular stock trading system with distinct core entities, functional types, and interfaces. At the center, the User entity acts as the primary connection point, interacting with subscriptions, portfolios, transactions, notifications, and watchlists. The model ensures that users can manage their investments efficiently while maintaining real-time stock tracking through API integrations. The Stock entity, another core component, connects to stock comparison, AI trend prediction, and API integration, ensuring data-driven decision-making. Additionally, interface types like IUserSession and IStockViewer support session management and stock visualization, enhancing the platform's usability. The composition relationships (diamonds) indicate ownership structures, ensuring that key elements like portfolios, watchlists, and notifications are user-specific, while stocks exist independently. This model is designed for flexibility and expansion, allowing future enhancements such as automated trading, advanced analytics, or AI-driven insights without disrupting the core architecture.

Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

Initial System Architecture



The Initial System Architecture diagram represents the foundational structure of the StockShark platform, showcasing key system components and their interactions. At the core, StockShark serves as the central system, connecting various interfaces such as iAI, iData, iLogin, iNotification, iPortfolio, iStockComparison, iStockData, iStockDisplay, and iStockGraph, ensuring modularity and extendability. The User entity is linked through the IUserSession interface, allowing authentication and session management, ensuring secure access to the system. Meanwhile, the Stock entity interacts with StockShark via the IStockViewer interface, enabling real-time stock data retrieval and visualization. This architecture provides a clear separation of concerns, ensuring scalability, maintainability, and integration of advanced functionalities like AI-based trend analysis and stock comparison. The structured design allows for future enhancements, such as automated trading or additional analytics, while keeping the system flexible and user-friendly.

Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

Unit Testing

What is Unit Testing?

Testing individual components (functions/methods) in isolation to ensure they behave as expected.

What did we do?

- Written Functions: Each function is a "unit" that can be tested. SQL for Testing Data: Used SQL queries to test data directly, bypassing the API with limited runs.
- GitHub Pull Requests: We worked on separate parts of the project and used pull requests for collaboration, code review, and merging.
- Manual Testing: Initially tested manually to verify functionality before committing changes.

Benefits we gained from this:

- Catch Bugs Early: Identify issues early without affecting the API.
- Refactor Confidently: Safely refactor code, knowing unit tests will ensure functionality remains intact.
- Collaboration and Code Review: Pull requests allowed for efficient collaboration and peer reviews.
- Automated Checks: SQL-based tests and unit tests ensure data integrity without hitting the API.

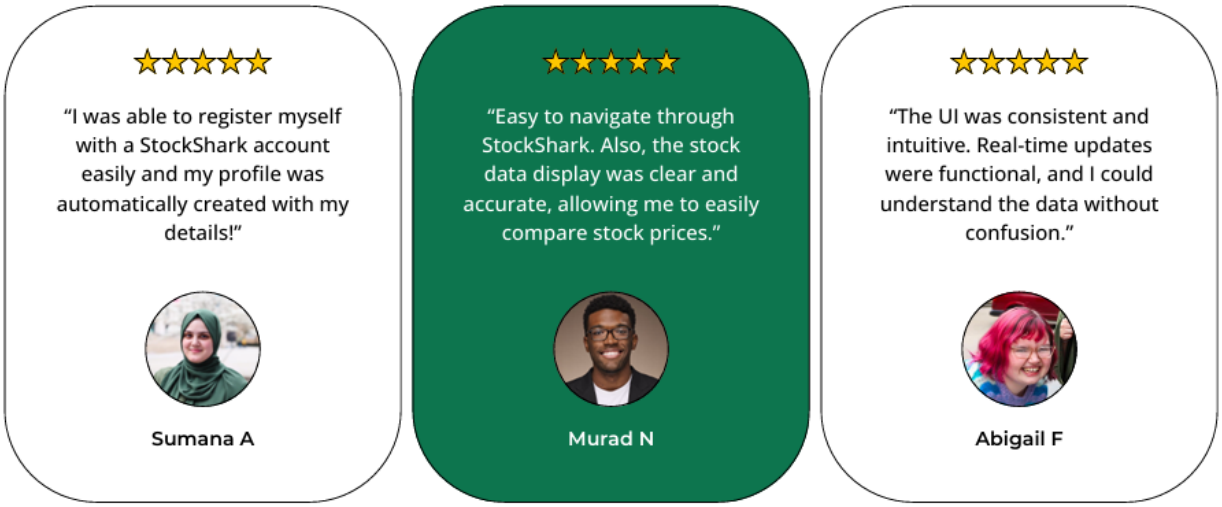
User Testing

What is User Testing?

Testing the application from an end-user perspective to ensure it works as intended and provides a smooth user experience

The core areas we focused on were key user flows such as registration, navigation, form submissions and viewing stock data. We tested our application on three different users from

three different backgrounds to gain a diverse set of end user feedback. Our results showed that all of them were happy to navigate across the pages as we kept our design simple, consistent and accessible to most. It also showed how registering and profile creation – core parts of user functionality, were easy to carry out. The data was also transferred correctly and readable as it was supposed to, and the graphs were displayed without any errors during these tests. Overall, the tests showed a general positive consensus, but in the future, we would like to gain broader feedback by carrying out many more user tests so that our results aren't limited. We would also like to integrate automated UI tests to ensure consistent performance across different user interactions.



Stock Shark

THE SOFTWARE ARCHITECTURE & DESIGN

Project Management

Code-Related Deliverables

Implementation of Clean Architecture Principles

- **Single Responsibility Principle (SRP):**
  Each class in the StockShark project is designed to have one responsibility. For example, servlets only manage HTTP requests/responses, while separate classes handle business logic and database interactions. This separation ensures that changes in one area (such as the API integration) do not impact others (e.g., user authentication).
- **Open/Closed Principle (OCP):**
  The system is built to be extendable without modifying existing code. For instance, new features can be added by extending base classes or interfaces rather than altering core logic. This allows us to implement future enhancements such as advanced chart interactivity and AI-based predictions without impacting current functionality.
- **Liskov Substitution Principle (LSP):**
  All subtypes and implementations are interchangeable with their base types. This means that classes implementing interfaces (like our Data_Handling_Component) can be substituted without altering the correctness of the program. This facilitates unit testing and future maintenance.
- **Interface Segregation Principle (ISP):**
  Clients are not forced to depend on methods they do not use. Interfaces in the project (for example, those defining user management functions and data handling operations) are focused and minimal. This ensures that each component only implements what is necessary, enhancing readability and maintainability.
- **Dependency Inversion Principle (DIP):**
  High-level modules do not depend on low-level modules; both depend on abstractions. The design uses abstract interfaces for key components (e.g., for database operations and API services), allowing us to change implementations without modifying the core business logic. This increases flexibility and supports unit testing by allowing dependency injection.

Component and Interface Implementation

- The **Database Client** (formerly DAO) encapsulates all database operations, ensuring that all SQL interactions are centralized and changeable without affecting business logic.
- **User Management and Data Handling Components** abstract complex operations (like authentication, portfolio management, and API integration) behind clean interfaces. These components satisfy the principles above while delivering modular and testable code.

Team Management

Team Coordination

- **Team Leader: Evan Balson**
  - **Responsibilities:** Directed Sprint 2 tasks, monitored progress, and ensured timely completion.
  - **Focus Areas:** System Interfaces, composite component diagrams, high-fidelity wireframes, and coding implementation for the Database, AI Analyzer, and API Service.
- **Scrum Master: Maahia Rahman**
  - **Responsibilities:** Scheduled weekly team meetings, managed the Business Concept Model, and created low-fidelity wireframes.
  - **Focus Areas:** Coding implementation for the Graphical Component and Stock Comparison Graph.
- **Mahbouba:**
  - **Responsibilities:** Managed the Business Type Model and participated in the documentation process.
  - **Focus Areas:** Coding implementation for the Notification System and Stock Comparison Session.
- **Hala:**
  - **Responsibilities:** Oversaw Use Case development, Initial Architecture, and defined Business Interfaces.
  - **Focus Areas:** Coding implementation for Portfolio, Login/Logout, and User modules. Also contributed to project documentation.